Lab 05: Local Search Algorithms

> **Objective:** Investigate how local search algorithms navigate a fitness landscape without retaining a search path. You will use the **Optimization Algorithm Simulator** from the lecture to observe how Hill Climbing, Simulated Annealing and Local Beam Search behave under different parameter settings, explain the trade-offs you observe, and then implement the generate–evaluate–prune cycle of Local Beam Search in Python. Total: **5.0 marks**.

Student Name	MRM AKKIL	**Student ID**	IT 24104314

Introduction

Local search algorithms do not build or store a path to the goal. They hold one current state (or a small set of states), look only at that state's immediate neighbours, and move according to a selection rule. Because the path is discarded, memory use is constant, which makes these methods suitable for very large or continuous state spaces where systematic search is infeasible. The cost of that efficiency is completeness: the algorithm can settle on a *local* optimum and stop.

> **Hill Climbing (Greedy):** always move to the best-scoring neighbour; halt when no neighbour improves on the current state.
> **Simulated Annealing:** accept an improving move always, and accept a worsening move with probability `P = e^(ΔE / T)`, where the temperature `T` falls over time according to a cooling schedule.
> **Local Beam Search:** keep *k* states, generate all their successors into one shared pool, and retain the best *k* of that pool for the next iteration.

Lab Activity

This is an individual activity. Complete each section on your own and type your answers directly into the boxes below.

Use the Optimization Algorithm Simulator provided in the lecture for Part A. Answer the MCQs in Part B directly in this sheet. For Part C, copy the skeleton code into your local IDE (PyCharm, VS Code, Jupyter or Colab), complete and test it, then paste the working code back into the box provided. Use **Print / Save as PDF** at the top of the page to export your completed sheet for submission.

Section 1: Part A — Simulation Analysis & Parameter Tweaks (2.0 Marks)

1. The Hill Climbing trap and step size (0.5 marks)

Set the simulator to **Hill Climbing (Greedy)** with **Starting X = 100**.
Tweak 1: run with **Step Size = 2** and note where the search becomes trapped.
Tweak 2: change **Step Size = 20** and run again. Does it step over the local maximum?

Run	Final X	Peak reached (local / global)
Step Size = 2	106	Local
Step Size = 20	512	Global

Explain the trade-off: how does a larger step size help the search escape a local optimum, and what is the dangerous drawback of setting the step size too large as the search nears the global maximum?

Escaping Local Optima: A larger step size increases the jump distance between evaluations, allowing the search algorithm to "step over" narrow local valleys/traps in the fitness landscape and reach the basin of attraction of a higher peak.
Dangerous Drawback: As the search nears the global maximum, a large step size causes the algorithm to overshoot the peak continuously, oscillating around it without converging precisely to the true optimum.

2. Initial condition sensitivity (0.5 marks)

Stay on **Hill Climbing** and set **Step Size = 5**.
Tweak 1: start at **X = 50**, run, and record the final peak.

Tweak 2: start at **X = 850**, run, and record the final peak.

Starting X	Final X	Final objective value
50	106	Local Peak Value
850	512	Global Peak Value

Observe & explain: compare the two final states. Why is greedy local search so sensitive to its initial configuration, and how does Random-Restart Hill Climbing mitigate this specific vulnerability?

Sensitivity: Greedy Hill Climbing only moves in the direction of strictly increasing fitness. Therefore, it is entirely constrained by its starting position's basin of attraction.
Random-Restart Mitigation: Random-Restart Hill Climbing runs multiple independent greedy searches initialized at randomly selected points across the state space. By sampling diverse starting locations, it dramatically increases the probability that at least one search begins within the basin of attraction of the global maximum.

3. Simulated Annealing: the cooling schedule (0.5 marks)

Switch to **Simulated Annealing** with **Starting X = 100** and **Initial Temp = 100**.
Tweak 1: Cooling Rate = 0.50 (fast cooling). Run the simulation.
Tweak 2: Cooling Rate = 0.99 (slow cooling). Run the simulation.

Cooling rate	Final X / value	Behaviour observed
0.50 (fast)	Stuck near local peak	
0.99 (slow)		

Explain the impact: how does the cooling rate control the balance between exploration (random walk) and exploitation (greedy climbing)? Referring to `P = e^(ΔE / T)`, state mathematically what happens to the acceptance probability for a worsening move when the temperature falls too rapidly.

Exploration vs. Exploitation: The cooling schedule controls the transition from random exploration (high temperature T) to greedy exploitation (low temperature T).
Mathematical Proof: In P = e ^ (Delta E) / (T), for a worsening move where Delta E < 0, when temperature T drops rapidly toward 0, the fraction Delta E / T becomes a large negative number. Consequently, e ^ -infty to 0, causing the acceptance probability P to plummet to zero instantly and forcing the algorithm to act purely as a greedy hill climber.

4. Local Beam Search: the loss of diversity (0.5 marks)

Set the simulator to **Local Beam Search** with **Beam Width (k) = 3** and run from a random start. You will most likely see all three agents converge onto the same peak within a few iterations, leaving the rest of the landscape unexplored.

Observe & explain: using the algorithm's successor-selection logic, explain mathematically why this loss of diversity occurs. In your answer, state how the successors of all *k* states are pooled and how the top *k* are chosen.

Loss of Diversity Reason: Local Beam Search evaluates and pools all successors generated by all k states into a single shared candidate list. It then selects the top k states purely based on fitness rank (reverse=True).
Mathematical Consequence: If one state finds a high-scoring peak, its generated successors quickly dominate the top k slots of the shared pool. As a result, all k beam states are drawn from the same neighborhood, causing the beam to collapse onto a single peak and abandon the rest of the landscape.

Section 2: Part B — Theoretical Foundations (MCQs) (1.5 Marks)

Select one option per question and type the letter in the answer box. 0.5 marks each.

5. Which statement best distinguishes Local Search from standard pathfinding (such as A*)?

A) Local search uses heuristics, while standard pathfinding does not.
B) Local search aims to find the optimal sequence of actions, while pathfinding only cares about the final state.
C) Standard pathfinding retains paths in memory to backtrack, whereas local search operates using only the current node (and its neighbours), focusing on the state rather than the path.
D) Local search algorithms are guaranteed to be complete and optimal in continuous spaces.

C

6. In Simulated Annealing the probability of accepting a worse state is `P = e^(ΔE / T)`. As the temperature *T* approaches 0, what happens to the behaviour of the algorithm?

A) It behaves exactly like a purely random walk.
B) It behaves exactly like greedy Hill Climbing, accepting only better states.
C) It accepts all states regardless of their fitness evaluation.
D) The algorithm terminates automatically because ΔE becomes undefined.

B

7. How does Stochastic Beam Search solve the loss-of-diversity problem found in standard Local Beam Search?

A) By choosing the next *k* states randomly, with probability proportional to their fitness, rather than strictly choosing the absolute best *k*.
B) By increasing the beam width *k* dynamically whenever a local minimum is detected.
C) By forcing agents to move in opposite directions (repulsion) when they get too close to each other.
D) By completely resetting the temperature whenever all *k* states converge.

A

Section 3: Part C — Guided Implementation: Local Beam Search (1.5 Marks)

Development instructions:

1. Copy the skeleton below into your local Python IDE (Jupyter, VS Code or Colab).
2. Complete the logic for **generating**, **evaluating** and **pruning** the neighbours.
3. Run the script. The `__main__` block contains a test case (a mathematical parabola); a correct implementation prints a success message.
4. Once it passes, paste your completed code into the box below.

Skeleton code

```
import random
```

```
def local_beam_search(initial_states, k, max_iters, objective_fn):
    """
    initial_states: list of k randomly generated starting positions
    k: the beam width (number of states to keep)
    objective_fn: the function we are trying to maximize
    """
    current_states = initial_states

    for iteration in range(max_iters):
        all_successors = []   # Will hold tuples of (state, fitness)

        # --- STEP 1, 2 & 3: Generate, Evaluate, and Pool Neighbors ---
        for state in current_states:
            # Generate 5 random neighbors for this specific state
            for _ in range(5):
                # HINT: neighbor = state + random.uniform(-10, 10)
                # [WRITE CODE HERE]

                # HINT: calc fitness -> fit = objective_fn(neighbor)
                # [WRITE CODE HERE]

                # HINT: append to pool -> all_successors.append((neighbor, fit))
                # [WRITE CODE HERE]
                pass

        # --- STEP 4: Prune the Beam (The Cut) ---
        # HINT 1: Sort the pool descending by fitness.
        #   all_successors.sort(key=lambda x: x[1], reverse=True)
        # [WRITE CODE HERE]

        # HINT 2: Slice the top 'k' items and extract just the state (index 0).
        #   current_states = [item[0] for item in all_successors[:k]]
        # [WRITE CODE HERE]

    return max(current_states, key=objective_fn)


# ======================================================================
# LOCAL TEST RUNNER - DO NOT MODIFY BELOW THIS LINE
# ======================================================================
if __name__ == "__main__":
    # Fix the random seed for reproducible testing
    random.seed(42)
```

Paste your completed, tested code here

```
import random

def local_beam_search(initial_states, k, max_iters, objective_fn):
    """
    initial_states: list of k randomly generated starting positions
    k: the beam width (number of states to keep)
    objective_fn: the function we are trying to maximize
    """
    current_states = initial_states

    for iteration in range(max_iters):
        all_successors = []  # Will hold tuples of (state, fitness)

        # STEP 1, 2 & 3: Generate, Evaluate, and Pool Neighbors
        for state in current_states:
            # Generate 5 random neighbors for this specific state
            for _ in range(5):
                neighbor = state + random.uniform(-10, 10)
                fit = objective_fn(neighbor)
                all_successors.append((neighbor, fit))

        # STEP 4: Prune the Beam (The Cut)
        # Sort the pool descending by fitness
        all_successors.sort(key=lambda x: x[1], reverse=True)

        # Slice the top 'k' items and extract just the state (index 0)
        current_states = [item[0] for item in all_successors[:k]]

    return max(current_states, key=objective_fn)


# ==========================================
# LOCAL TEST RUNNER DO NOT MODIFY BELOW THIS LINE
# ==========================================
if __name__ == "__main__":
    # Fix the random seed for reproducible testing
    random.seed(42)

    # Test Objective Function: Inverted parabola with peak at x = 5.0
    def target_function(x):
        return -(x - 5.0) ** 2 + 100.0
```

```
    k_width = 3
    starts = [random.uniform(-50, 50) for _ in range(k_width)]
    best_found = local_beam_search(starts, k=k_width, max_iters=50,
objective_fn=target_function)

    print(f"Test Completed Successfully!")
    print(f"Best State Found: {best_found:.4f}")
    print(f"Objective Value: {target_function(best_found):.4f}")
```

Test output

Paste the exact output printed by the test runner, including the success message and the best state returned.

```
Test Completed Successfully!
Best State Found: 4.9998
Objective Value: 100.0000
```

Discussion

Local Beam Search with $k = 3$ is not the same as running Hill Climbing three times independently. Explain the difference and say which one preserves more diversity, and why.

Difference: Independent Hill Climbing runs 3 isolated searches that never communicate. Local Beam Search pools successors from all k states together, allowing information sharing across beams.
Diversity: Running Hill Climbing 3 times independently **preserves more diversity**. In Local Beam Search, if one beam state discovers a very high fitness area, its offspring quickly fill all k slots, causing all 3 search paths to converge onto a single peak. Independent runs prevent this collapse.

Your objective function is expensive to evaluate and you are given a fixed budget of evaluations. Which algorithm from this lab would you choose, and how would you set its parameters? Justify your choice using your Part A observations.

Choice: Simulated Annealing with a high initial temperature and a moderate cooling rate (e.g., 0.95).

Justification: Local Beam Search evaluates 5 * k neighbors per iteration, consuming the budget rapidly while risking beam collapse. Simulated Annealing evaluates only **one** candidate neighbor per step, maximizing the depth and quality of exploration per function call.